Next.js Preparation Guide

Nov 9, 2026 · @restocare

Introduction

Next.js is a React framework for building full-stack web apps. React gives you components. Next.js adds routing, server rendering, data fetching, bundling and optimisation, so you don't have to wire them up yourself.

This guide targets **Next.js 16** (the version in this project, 16.4.3) with **React 19** and the **App Router**.

Key features:

- **File-system routing**: folders inside `app/` become URLs.
- **Server Components by default**: components render on the server and send less JavaScript to the browser.
- **Multiple rendering modes**: static (built once), dynamic (per request), and streaming with `loading.tsx` and Suspense.
- **Built-in data fetching and caching**: `async` components, `fetch`, Server Functions and revalidation.
- **Route Handlers**: write API endpoints inside the same project (`route.ts`).
- **Optimisations**: `next/image`, `next/font`, `next/link` prefetching, metadata for SEO.
- **Turbopack**: the default bundler for `next dev` and `next build`.

React vs Next.js in one line: React is the UI library; Next.js is the framework that decides how that UI is routed, rendered and served.

Uses

Pick Next.js when a React app needs SEO, fast first load, or a backend in the same codebase.

Use case	Why Next.js fits
Marketing sites, landing pages, blogs	Static rendering + metadata API give fast, SEO-friendly pages
E-commerce and product catalogues	Server rendering, image optimisation, revalidation for changing prices and stock

Use case	Why Next.js fits
SaaS dashboards and admin panels	Layouts, protected routes, Server Functions for forms and mutations
Full-stack apps (backend-for-frontend)	Route Handlers and Server Functions call databases and APIs without a separate server
Docs and content sites	MDX support, static generation, fast navigation
Multi-language or multi-tenant sites	Dynamic segments and `proxy.ts` for locale or tenant routing

When plain React (for example Vite) may be enough:

- A fully client-side app behind login where SEO doesn't matter.
- A small widget embedded in another site.
- You already have a separate backend and need only a static SPA.

How to use

You need Node.js 20.9 or newer. One command creates a ready project with TypeScript, Tailwind CSS, ESLint, App Router and Turbopack.

1. Create a project

```
npx create-next-app@latest my-app --yes
cd my-app
npm run dev
```

Open `http://localhost:3000`. `--yes` skips the prompts and uses the recommended defaults (import alias `@/*`).

2. Scripts

Command	What it does
`npm run dev` (`next dev`)	Development server with Turbopack and hot reload
`npm run build` (`next build`)	Production build
`npm run start` (`next start`)	Runs the production build
`npm run lint` (`eslint`)	Lints the code

Turbopack is the default bundler. Use `next dev --webpack` only if you need Webpack.

3. Project structure

```
my-app/
  app/                App Router: routes live here
    layout.tsx        Root layout (required, has <html> and <body>)
    page.tsx          Home page  ->  /
    globals.css
  public/             Static files served from /  (images, favicon)
  next.config.ts      Next.js configuration
  proxy.ts            Optional request proxy (was middleware.ts)
  .env.local          Local environment variables (don't commit)
  tsconfig.json
  package.json
```

You can also put `app/` inside an optional `src/` folder, as this project does (`src/app`, `src/components`).

4. Root layout and first page

```
// app/layout.tsx
export default function RootLayout({ children }: { children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  )
}
```

```
// app/page.tsx
export default function HomePage() {
  return <h1>Hello, Next.js!</h1>
}
```

5. Navigate between pages

Use `<Link>` from `next/link`, not `<a>`, for internal links. It navigates on the client and prefetches the route.

```
import Link from 'next/link'

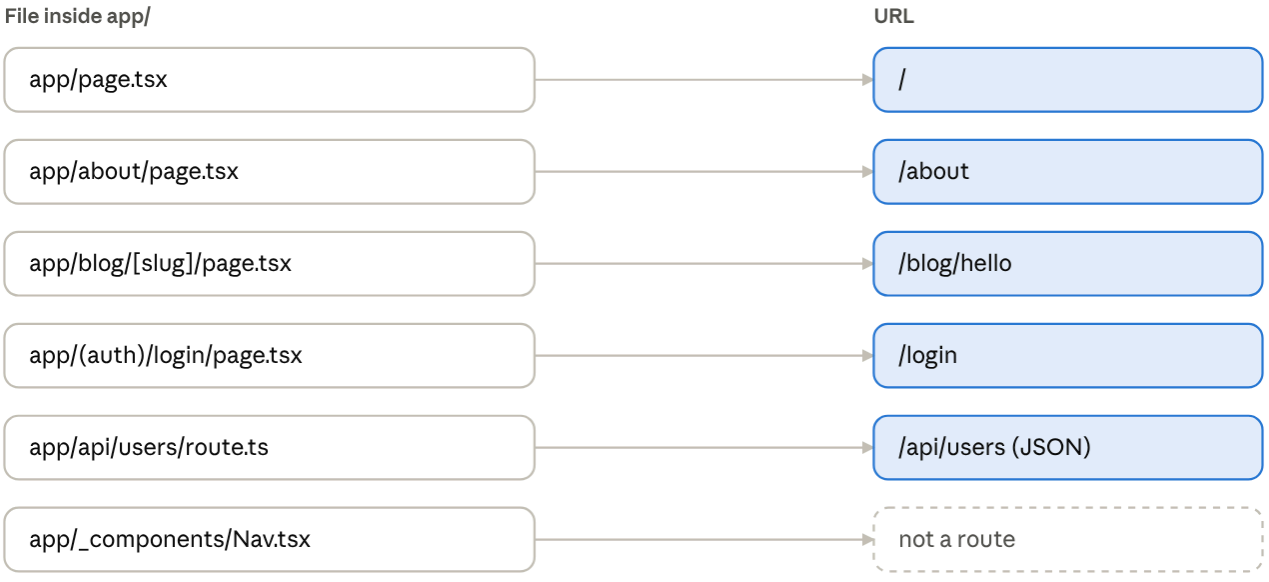
<Link href="/about">About</Link>
```

For navigation in code (inside a Client Component), use `useRouter` from `next/navigation`: `router.push('/dashboard')`.

Folder-based routing

In the App Router, every folder inside `app/` is a URL segment, and a folder becomes a public route only when it contains `page.tsx` (or `route.ts` for an API).

A folder becomes a URL only where page.tsx or route.ts exists



[slug] is a variable segment, (auth) is left out of the URL, _components never becomes a route.

app/ folders to URLs · 6 examples

The folder path is the URL path, except for brackets, parentheses and underscores, which change how a segment is matched.

Basic and nested routes

File	URL
`app/page.tsx`	`/`
`app/about/page.tsx`	`/about`
`app/blog/page.tsx`	`/blog`
`app/blog/authors/page.tsx`	`/blog/authors`
`app/dashboard/settings/page.tsx`	`/dashboard/settings`

A folder without `page.tsx` is not reachable. So you can keep components and helpers next to the route safely.

Special files

File	Purpose
`page.tsx`	The UI for that URL; makes the route public
`layout.tsx`	Shared UI (header, sidebar) that wraps the segment and its children; keeps state on navigation
`template.tsx`	Like a layout, but re-mounts on every navigation
`loading.tsx`	Loading skeleton, shown automatically through a Suspense boundary
`error.tsx`	Error boundary for the segment (must be a Client Component)
`global-error.tsx`	Error boundary for the root layout
`not-found.tsx`	UI for `notFound()` and unmatched URLs
`route.ts`	API endpoint (GET, POST, …) — no UI
`default.tsx`	Fallback for parallel routes

Render order inside one segment: `layout` → `template` → `error` → `loading` → `not-found` → `page`. A child segment renders inside its parent's layout.

Layouts

```
// app/dashboard/layout.tsx  ->  wraps /dashboard and everything below it
export default function DashboardLayout({ children }: { children:
React.ReactNode }) {
  return (
    <section>
      <nav>Sidebar</nav>
      {children}
    </section>
  )
}
```

Dynamic routes

Use square brackets when part of the URL is a variable.

Folder	Matches	`params`
`app/blog/[slug]/page.tsx`	`/blog/hello`	`{ slug: 'hello' }`
`app/shop/[...slug]/page.tsx`	`/shop/a`, `/shop/a/b`	`{ slug: ['a', 'b'] }`
`app/docs/[[...slug]]/page.tsx`	`/docs`, `/docs/a/b`	`{}` or `{ slug: ['a', 'b'] }`

In Next.js 16, `params` and `searchParams` are **Promises** — always `await` them.

```
// app/blog/[slug]/page.tsx
export default async function Page(props: PageProps<'/blog/[slug]'>) {
  const { slug } = await props.params
  return <h1>Post: {slug}</h1>
}
```

`PageProps`, `LayoutProps` and `RouteContext` are global type helpers; no import needed.

Route groups and private folders

Folder	Effect
`(marketing)/`	Groups routes without adding to the URL. `app/(marketing)/about/page.tsx` → `/about`

Folder	Effect
`(auth)/layout.tsx`	Gives one group its own layout (e.g. login/signup without the app header)
`_components/`	Private folder; never becomes a route. Good for colocated UI and utils

Advanced patterns

Pattern	Meaning	Typical use
`@slot`	Parallel route (named slot in a layout)	Dashboard with sidebar + main panels
`(.)folder`	Intercept a sibling route	Open a photo in a modal over the list
`(..)folder`	Intercept a route one level up	Overlay from a nested page
`(...)folder`	Intercept from the root	Login modal from anywhere

API routes (Route Handlers)

```
// app/api/users/[id]/route.ts  ->  GET /api/users/42
import type { NextRequest } from 'next/server'

export async function GET(_req: NextRequest, ctx:
RouteContext<'/api/users/[id]'>) {
  const { id } = await ctx.params
  return Response.json({ id })
}
```

A folder can't have both `page.tsx` and `route.ts` at the same level.

Server vs Client Components and data fetching

Every component in `app/` is a Server Component unless its file starts with `'use client'`. Keep pages server-side and push `'use client'` down to the small interactive parts.

	Server Component (default)	Client Component (`'use client'`)
Runs on	Server only	Server for first HTML, then browser
Can be `async` and fetch data directly	Yes	No (use a library such as SWR/React Query, or `use()`)
Can use secrets, DB, env vars	Yes	No (only `NEXT_PUBLIC_*` vars)
`useState`, `useEffect`, event handlers	No	Yes
Browser APIs (`window`, `localStorage`)	No	Yes
JS sent to the browser	None for the component	Component + its imports

`'use client'` marks a boundary: everything that file imports also goes to the client bundle. A Server Component can pass a Server Component to a Client Component as `children`.

Fetching data on the server

```
// app/posts/page.tsx
export default async function Page() {
  const res = await fetch('https://api.example.com/posts')
  const posts = await res.json()
  return <ul>{posts.map((p) => <li key={p.id}>{p.title}</li>)}</ul>
}
```

`fetch` is **not cached by default** in Next.js 16. Add `loading.tsx` or wrap slow parts in `<Suspense>` to stream the rest of the page first.

Caching (Cache Components)

With `cacheComponents: true` in `next.config.ts`, opt in to caching with the `'use cache'` directive on a function, component or page.

```
import { cacheLife, cacheTag } from 'next/cache'

export async function getProducts() {
  'use cache'
  cacheLife('hours')
  cacheTag('products')
  return db.product.findMany()
}
```

API	Use it to
`cacheLife('hours')`	Set how long a cached result lives
`cacheTag('products')`	Label cached data so you can clear it later
`revalidateTag('products', 'max')`	Mark tagged data stale (stale-while-revalidate)
`updateTag('products')`	Expire and refresh immediately (read-your-own-writes, in Server Actions)
`revalidatePath('/products')`	Revalidate one route

Mutating data (Server Functions)

```
// app/actions.ts
'use server'
import { revalidatePath } from 'next/cache'

export async function createPost(formData: FormData) {
  await db.post.create({ data: { title: formData.get('title') as string } })
  revalidatePath('/posts')
}
```

```
// use it directly in a form, no API route needed
<form action={createPost}>
  <input name="title" />
  <button type="submit">Save</button>
</form>
```

Interview and prep questions

1. **What is Next.js and how is it different from React?** React is a UI library. Next.js is a React framework that adds routing, server rendering, data fetching, API routes and build optimisation.

2. **App Router vs Pages Router?** App Router (`app/`) uses layouts, Server Components, streaming and Server Functions; it is the recommended default. Pages Router (`pages/`) is the older model with `getServerSideProps` / `getStaticProps`.
3. **How does file-based routing work?** Each folder in `app/` is a URL segment; `page.tsx` makes it public; `layout.tsx` wraps it and its children.
4. **How do you create a dynamic route?** Name the folder `[id]`, then read `const { id } = await params` in the page. Use `[...slug]` for catch-all and `[[...slug]]` for optional catch-all.
5. **What is a route group and why use it?** A folder in parentheses, e.g. `(auth)`. It organises routes and gives them a separate layout without changing the URL.
6. **Server Component vs Client Component?** Server Components run only on the server, can be `async` and send no JS. Client Components (`'use client'`) can use state, effects and browser APIs.
7. **Layout vs template?** A layout keeps its state across navigation; a template re-mounts on every navigation.
8. **How do `loading.tsx` and `error.tsx` work?** `loading.tsx` wraps the page in Suspense and shows a fallback while it streams. `error.tsx` is a Client Component error boundary with a `reset` function.
9. **What are Server Functions (Server Actions)?** Async functions marked `'use server'` that run on the server and can be called from forms or event handlers, so you don't need an API route for mutations.
10. **How does caching work in Next.js 16?** `fetch` is not cached by default. With Cache Components you opt in using `'use cache'`, set lifetime with `cacheLife`, and clear with `cacheTag` + `revalidateTag` / `updateTag` or `revalidatePath`.
11. **What is `proxy.ts`?** The Next.js 16 name for `middleware.ts`. It runs before a request is handled, for redirects, rewrites, auth checks or locale detection.
12. **Why use `next/link`, `next/image` and `next/font`?** `Link` gives client-side navigation and prefetching; `Image` resizes and lazy-loads images; `font` self-hosts fonts with no layout shift.
13. **How do you add SEO metadata?** Export a `metadata` object or a `generateMetadata` function from a `page` or `layout`. Add `sitemap.ts` and `robots.ts` in `app/`.
14. **Static vs dynamic rendering?** Static routes render at build time and are served from cache. A route becomes dynamic when it reads request data such as `cookies()`, `headers()` or `searchParams`.
15. **Which environment variables reach the browser?** Only those prefixed with `NEXT_PUBLIC_`. Others stay on the server.

Quick revision checklist

- [ ] Create a project with `create-next-app` and run `dev`, `build`, `start`
- [ ] Build nested routes, a shared layout and a route group
- [ ] Build a dynamic `[slug]` page and `await params`
- [ ] Add `loading.tsx`, `error.tsx` and `not-found.tsx`
- [ ] Fetch data in a Server Component; add one Client Component with state
- [ ] Write a Route Handler and a Server Function form
- [ ] Cache a function with `'use cache'` and revalidate it